

How to train the language models

This contains a compilation of some thoughts I have around training language models to predict the keep/remove decisions.

We've already trained logistic regression and XGBoost-based models, across a few ablations. Now I'd like to see how language models would fare.

Some variants I'm thinking of include:

- One-shot prompting:
 - With a small model
 - With a large model
- Few-shot prompting:
 - With a small model
 - With a large model
- ModernBERT
- LoRA fine-tuning

I'll also do the prompting ones across two sets of inputs:

- Just the original post
- Original post + mirrored post

We showed users both the original post and the mirrored post as part of the linked fate procedure, and we saw a significant difference in keep/remove decisions based on whether users were in the linked fate condition or if they only saw the original post. However, when we trained our predictive models, we saw that including only the original post was just as predictive as showing both the original and mirrored posts, suggesting that it was the mere presence of the mirrored post that affected whether users chose to keep or remove. We also observed that users in our training condition (where they saw the linked fate procedure for the first half of the study, and only the original posts in the second half of the study) also matched the performance of users in the linked-fate-only condition, censoring outgroup posts less and ingroup posts more as compared to users in the control condition (i.e., only saw the original posts). All of this suggests that the key driver is the mere presence or awareness of the linked fate procedure (i.e., seeing the mirrored posts).

For completeness, we can do some initial experiments that include both the original post and the original post plus the mirrored post. However, I expect those to have non-significant results due to our previous findings suggesting that the mere presence of the mirror, rather than anything about the substance of the mirrored post itself, drove performance. If we replicate that, then we'll continue on training the ModernBERT and LoRA-tuned models with just the original text.

Experiment 1: Prompting

For the prompting experiments, we'll do them in `models/llm_api/{one_shot/few_shot}/{original/original_plus_`

Each will have a `prompt.py` and a `train.py`, for consistency with the other approaches.

For the models, we'll use:

- small: gpt-5.4 nano
- large: gpt-5.5

We'll run these and report the results as a table here, with columns:

- type: one-shot/few-shot
- ablation: original, original plus mirror
- model size: small, large
- model name
- split: train/test
- accuracy, other metrics...

Prompting results (Experiment 1)

We only completed full-dataset runs for the two one-shot / small (**gpt-5.4-nano**) ablations (original-only and original-plus-mirror). Each run scored the full 80/20 split (`n_train=7032`, `n_test=1759`; 8,791 API requests per variant). We did not run the remaining six variants (one-shot/large, few-shot/{original,original_plus_mirror}x{small,large}) for cost reasons; the two completed runs are enough of a prompting baseline to compare against the embedding classifiers and to decide whether to invest in ModernBERT / LoRA next.

Precision, recall, F1, ROC-AUC, and PR-AUC use **remove** as the positive class (`y=1`).

type	ablation	model_size	model_name	split	accuracy	precision	recall	f1	roc_auc	pr_auc
one-shot	original	small	gpt-5.4-nano	train	0.690	0.519	0.443	0.478	0.678	0.491
one-shot	original	small	gpt-5.4-nano	test	0.686	0.510	0.485	0.497	0.695	0.515
one-shot	original plus mirror	small	gpt-5.4-nano	train	0.682	0.504	0.308	0.382	0.627	0.448
one-shot	original plus mirror	small	gpt-5.4-nano	test	0.676	0.490	0.314	0.383	0.615	0.448

On the test set, original-only one-shot prompting slightly outperforms original-plus-mirror (accuracy 0.686 vs 0.676; F1 0.497 vs 0.383), consistent with the embedding ablations where adding mirror text did not help. Absolute performance is in the same ballpark as logistic regression on original-only embeddings (test accuracy ~ 0.67 , F1 ~ 0.55), so prompting alone is not a clear win over the cheaper embedding baselines.

Experiment 2: ModernBERT

We also want to use ModernBERT to develop a fine-tuned classifier for our task. ModernBERT is an improvement over the original BERT model, with additions like RoPE, FlashAttention, and other techniques from language model research. The 8,192-token context window is a big practical improvement over classic BERT’s usual 512-token limit.

HuggingFace natively supports ModernBERT. In addition, we can use AWS Sagemaker as our trainer in order to avoid having to do local computation. We’ll use the base model of ModernBERT, not the large one.

We’d like to use the following:

- Weights and Biases for ML training logging.
- HuggingFace for the interface.
- AWS Sagemaker for the compute.

We’ll use the same training data that we used for our other ML models (where each row is 1 post and label). We’ll use only the original text, rather than the original text plus the mirrored text.

We’ll collect the following metrics:

- Accuracy
- Precision
- Recall
- F1

We’ll use cross-entropy loss with 2 labels. Because of our class imbalance, we’ll use weighted loss, where we multiply the loss by the class weight. We want to upweight the minority class (`remove=1`) so we prioritize getting those correct. We’ll double the loss for `remove=1`. This also lets us improve recall for `remove` labels, but we’ll want to be careful as we also don’t want many false positives (which worsens our precision).

Our prediction task is “predict remove”, so we need to transform the training data to create a binary label.

We’ll develop in the following order:

Step 1: Head-only training (frozen encoder)

Fine-tune ModernBERT on the training data and evaluate training curves. We'll evaluate the training curves to review for overfitting.

We'll do head-only training for our task, as our training dataset is pretty small and we want something that is somewhat able to work out-of-distribution and avoid catastrophic forgetting.

Step 2: Calibration

Given our outputs, we can evaluate calibration curves varying the binary thresholds from $p=0.1$ to $p=0.9$, in increments of 0.1.

More implementation details

We'll store all the work in `experiments/predict_keep_remove_2026_07_01/models/modernbert/`

We'll use the following file structure:

```
experiments/
  predict_keep_remove_2026_07_01/
    dataloader.py          # existing raw dataframe loader

    models/
      modernbert/
        README.md
        dataloader.py      # wraps parent dataloader, converts labels
        train.py           # local/SageMaker-compatible training entrypoint
        evaluate.py        # optional test-set evaluation
        predict.py         # optional inference helper
        launch_sagemaker.py # submits remote SageMaker job
        requirements.txt
        configs/
          modernbert_base.yaml
        artifacts/
          .gitkeep         # local outputs ignored by git
```

For packages, add to `requirements.txt` and then also add to the root `pyproject.toml` as optional dependencies titled “modernbert-training”.

We can use something like this as the config:

```
# experiments/predict_keep_remove_2026_07_01/models/modernbert/configs/modernbert_base.yaml

model_name: answerdotai/ModernBERT-base

text_col: text
label_col: label
```

```

max_length: 256
learning_rate: 2e-5
num_train_epochs: 10 # let's run for 10 epochs
per_device_train_batch_size: 8
per_device_eval_batch_size: 16
weight_decay: 0.01

```

```

output_dir: artifacts/modernbert-base
random_state: 42

```

For ~2,000 posts and 10 epochs, we can use a smaller GPU setup:

- Instance: ml.g4dn.xlarge
- GPU: 1 x NVIDIA T4, 16 GB GPU memory
- Estimated SageMaker on-demand price: about \$0.74/hr in us-east-2
- Expected runtime: ~5-20 minutes
- Expected compute cost: usually <\$0.25, likely closer to \$0.05-\$0.15

Let's use `us-east-2` for our AWS setup.

For our data splits, we want to do a 80/10/10 split between train/validation/test.

ModernBERT results (Experiment 2)

Full 10-epoch head-only fine-tune of `answerdotai/ModernBERT-base` on original post text only (`freeze_encoder=true`, class weights `keep=1.0 / remove=2.0`, `max_length=256`). Trained on SageMaker `ml.g4dn.xlarge` in `us-east-2` (job `modernbert-keep-remove-2026-07-03-21-09-05-077`; run id `2026_07_03-210901`). Split sizes: `n_train=7032`, `n_val=879`, `n_test=880`. Metrics below use decision threshold 0.5 with `remove` as the positive class (`y=1`).

Artifacts: `experiments/predict_keep_remove_2026_07_01/models/modernbert/artifacts/modernbert-b`
`(metrics.json, predictions, calibration.json)`. S3: `s3://jspsych-mirror-view-4/modernbert-training/`
W&B: `modernbert-base-2026_07_03-21:18:33`.

model	freeze_encoder	split	accuracy	precision	recall	f1	roc_auc	pr_auc
ModernBERT-true base		train	0.691	0.515	0.585	0.548	0.719	0.545
ModernBERT-true base		val	0.681	0.502	0.552	0.525	0.693	0.525
ModernBERT-true base		test	0.694	0.520	0.596	0.555	0.742	0.569

On the test set, head-only ModernBERT reaches accuracy 0.694 and F1 0.555, slightly above one-shot original-only prompting (accuracy 0.686, F1 0.497) and in line with the logistic-regression embedding baseline (test accuracy ~0.67, F1

~0.55). Train and val metrics are close to test (no large train-test gap), consistent with a frozen encoder and a small trainable head (1,538 parameters). Threshold calibration over 0.1-0.9 is in `calibration.json` for follow-up threshold selection.

ModernBERT threshold tuning

Test-set threshold sweep (0.1-0.9, step 0.1) on run 2026_07_03-210901. Artifacts: `models/modernbert/outputs/threshold_analysis/2026_07_05-13:57:45/`.

metric	highest value	value at p=0.5	threshold at highest
accuracy	0.726	0.694	0.6
precision	0.741	0.520	0.8
recall	0.996	0.596	0.1
f1	0.578	0.555	0.4

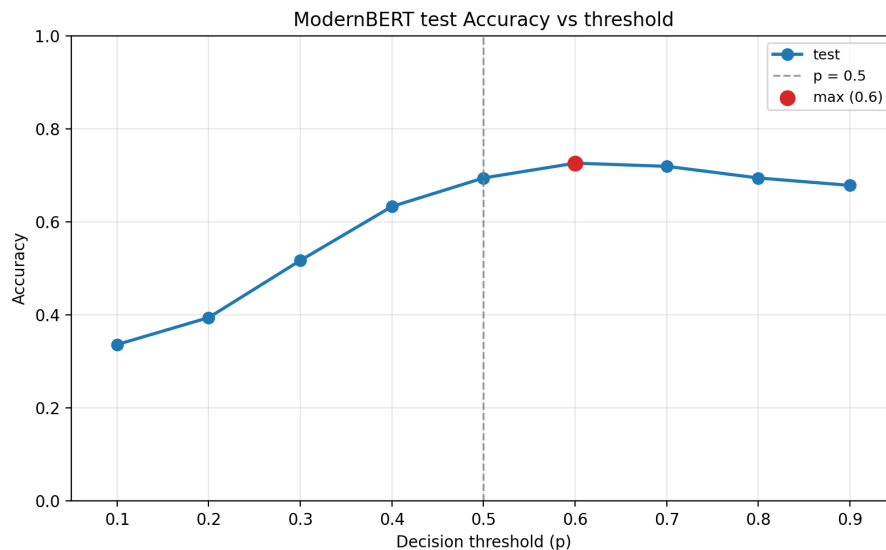


Figure 1: ModernBERT test accuracy vs threshold

From this experimentation, it seems like using a default threshold of $p=0.5$ does, on average, generally the best across all metrics.

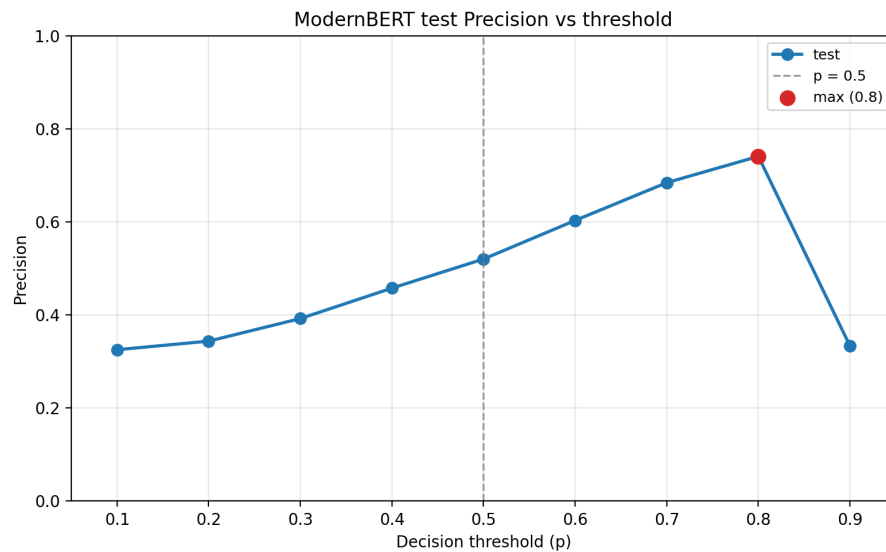


Figure 2: ModernBERT test precision vs threshold

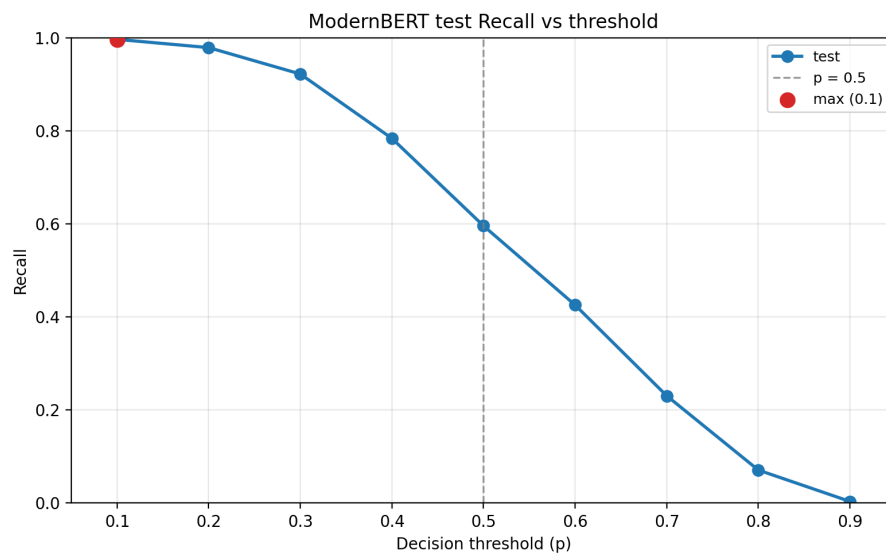


Figure 3: ModernBERT test recall vs threshold

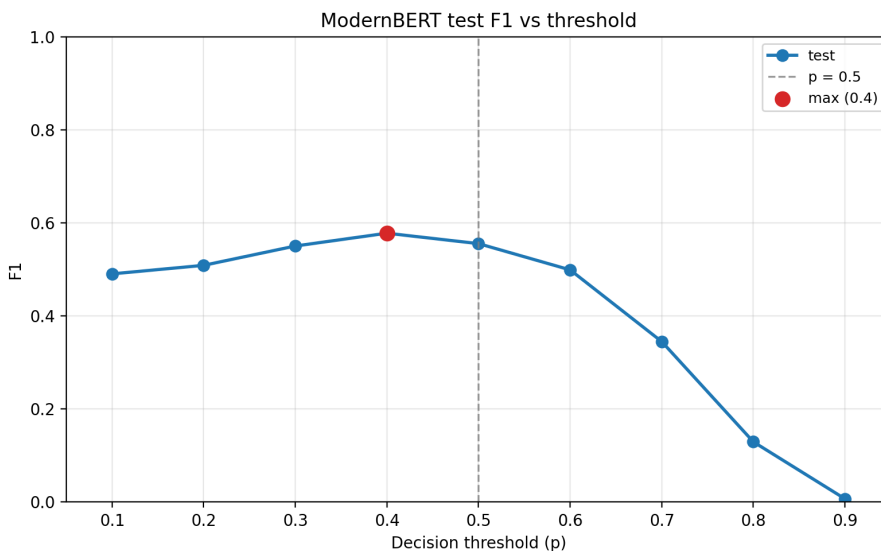


Figure 4: ModernBERT test F1 vs threshold

Experiment 3: Tuning an LLM

Motivation for Experiment 3

Even though technically we can build explainable models using the combination of embeddings and linear models, these are not going to be as straightforward for interpretation purposes as a language model. For embeddings, we have to interpret them through approaches like clustering and dimensionality reduction. However, for a large language model, we can directly use tools such as attention maps. In addition, we can interpret models like tree models or regression models through measures of feature importance, but the individual features for a 256-dimensional embedding are not so easy to interpret. We also have the additional benefit that if we train a generative model, we can have a result that's easily usable for generative purposes. We can do something as simple as asking the model what it's looking at to be able to make certain decisions. This also drives our next order goal of figuring out a set of factors or features or policies that people are using in their interpretation and determination of content moderation policies. We again could, in theory, do all this through some workarounds related to manual feature engineering and looking at the feature importance of traditional models such as regression and tree-based models. However, it's much more straightforward for the stakeholder interpretability story, perhaps less for the robustness story, but it definitely is a compelling user experience to be able to ask a fine-tuned model what exactly it is using to make a certain set of decisions.

Design for Experiment 3

For the setup here, I'm thinking that we can pass in both the original and the mirror post, but we can't tell the model which is which. We'll just say to consider these two sets of posts and have the model itself do the same mirror view task as humans.

We can see how it does out of the box. We can also check that against how it does after fine-tuning. We can do this for the same model to see how it does.

We'll use the following prompt, matching what we show in the actual study:

We are developing a new social media platform designed specifically for political discussion

The pairs are political mirrors of each other. This means that the mirror text recreates the

Original Text:

I'm a bleeding-heart liberal, and I think the issue of abortion is obviously about protecting

Mirror Text:

I'm a staunch conservative, and abortion is fully about the sanctity of human life before b

Notice that the mirror text changes the core message to match that of the opposite political

Your job is to decide whether both posts in the pair should be allowed or removed from the p

When making your decisions, consider generally whether a post contributes to a healthy environ

Post 1: {post_1_text}

Post 2: {post_2_text}

Allow Or Remove?

We want to use a Pydantic model for this, a pydantic model, IsRemoveResult, with one value, is_remove, a boolean.

We'll shuffle the original and mirrored text randomly within the prompt (e.g., sometimes the original post will be Post 1, sometimes it'll be Post 2).

Step 1: Zero-shot baseline We'll use some open source models exposed via Bedrock API as a baseline. Here's a synopsis of the models I'd like to try and what ChatGPT says about them:

Pick	Bedrock run- time model ID for base- line in- fer- ence	Size / architecture	Why I'd consider it for your path	Weights- only mem- ory to “hold the model”	Estimated LoRA / QLoRA fine-tuning requirement	Single- H100 ver- dict
Mistral Minis- tral 3 8B In- struct	Mistral 8B Instruct active Bedrock model, 128K context, 8K max output. (AWS Docu- mentation)	Mistral baseline: small enough to fine-tune cheaply, large enough to be more credible than a 3B model for nuanced classifica- tion/general instruction tasks.	BF16 weights ~ 16GB ; 8-bit ~ 8GB ; 4-bit ~ 4- 5GB before run- time over- head.	Comfortable. BF16 LoRA likely fits on 1xH100; QLoRA likely fits on much smaller GPUs too. Estimate: ~18-30GB VRAM for QLoRA depending on sequence length/packing; low H100-hour cost for a small dataset.	Strong yes. Best “move fast” op- tion.	

Pick	Bedrock run- time model ID for base- line in- fer- ence	Size / architecture	Why I'd consider it for your path	Weights- only mem- ory to "hold the model"	Estimated LoRA / QLoRA fine-tuning requirement	Single- H100 ver- dict
Mistral Minis- tral 14B 3.0 In- struct	Mistral 14B Instruct	active Bedrock model, 128K context, 8K max output. (AWS Docu- mentation)	Bedrock- instruct qual- ity/size tradeoff than 8B while still very practical for adapter fine-tuning. I'd use this as the main Mistral candidate if Bedrock baseline looks promising.	BF16 weights ~ 28GB ; 8-bit ~ 14GB ; 4-bit ~ 7- 9GB before over- head.	Comfortable to moderate. BF16 LoRA should fit on 1xH100 with sane context/batch; QLoRA is easy. Estimate: ~ 28-45GB VRAM for QLoRA; maybe ~ 0.5-2x the H100 time of the 8B model.	Strong yes. My pre- ferred Mis- tral pick.

Pick	Bedrock run- time model ID for base- line in- fer- ence	Size / architecture	Why I'd consider it for your path	Weights- only mem- ory to “hold the model”	Estimated LoRA / QLoRA fine-tuning requirement	Single- H100 ver- dict
Qwen3.1-72B-Base, v1 32B	active Bedrock model, 32K context, 8K max output, reasoning supported. AWS names this without an Instruct suffix, but it is exposed through chat/converse- style Bedrock APIs. (AWS Documenta- tion)	Best Qwen general- purpose candidate that is still realistic on one H100. Dense ar- chitecture also makes fine-tuning behavior simpler and more pre- dictable than MoE.	BF16 weights ~ 64GB; 8-bit ~ 32GB; 4-bit ~ 18- 22GB with quanti- zation over- head.	QLoRA strongly recommended. BF16 LoRA is too tight once you add activa- tions/KV/optimizer overhead. Estimate: ~55-75GB VRAM for QLoRA with short/moderate context and careful settings.	Yes, but use QLoRA. My pre- ferred Qwen pick.	

Pick	Bedrock run- time model ID for base- line in- fer- ence	Size / architecture	Why I'd consider it for your path	Weights- only mem- ory to "hold the model"	Estimated LoRA / QLoRA fine-tuning requirement	Single- H100 ver- dict
	Qwen3	80B total - 80B active	Next 80B A3B MoE ; active Bedrock model, 256K context, 8K max output, reasoning supported. (AWS Documentation)	BF16 weights ~ 160GB , so not one H100 in BF16; 8-bit ~ 80GB before over-head; 4-bit ~ 45-55GB before over-head.	Aggressive QLoRA only. Possible on 1xH100 only with optimized MoE support, short context, small microbatch, and careful adapter target selection. Estimate: ~ 65-80GB+ VRAM ; otherwise use 2x80GB for headroom. Unsloth specifically documents Qwen3 and Qwen3 MoE fine-tuning support, though exact VRAM depends heavily on model/config. (Unsloth - Train and Run Models Locally)	Borderline. Good Bedrock base-line, risky as your first fine-tune target.

We'll put this in `experiments/predict_keep_remove_2026_07_01/models/llm_finetuning/api_baselines/{name}/`. Within each, we'll have a `prompt.py` and a `train.py`. We'll keep each model's results in its `api_baselines/{model name}/outputs/` folder. We also want a baseline `api_baselines/` folder with common scripts such as a `runner.py`. We'll want to follow a lot of the same patterns available in `modernbert` and `llm_api` as to what files we need and how to structure them.

We'll also use the `dataloader.py` from `experiments/predict_keep_remove_2026_07_01` to grab the data. We'll just label all the posts and then evaluate accuracy, precision, recall, and F1. Then we'll have an `api_baselines/plot_results.py` that creates JSON/PNG results for each metric, using output patterns and graphs similar to `experiments/predict_keep_remove_2026_07_01/models/modernbert/threshold_analysis.py`.

Step 1 results (Bedrock zero-shot baseline) We ran all four Bedrock models on the **full dataset** (8,791 pairs; study linked-fate prompt with blinded Post 1/Post 2 shuffle; hard-label metrics with **remove** as the positive class). Artifacts live under `experiments/predict_keep_remove_2026_07_01/models/llm_finetuning/api_baselines/{model_id}`

model	bedrock_model_id	accuracy	precision	recall	f1
Minstral 3 14B Instruct	mistral.ministral-3-14b-instruct	0.689683	0.512244	0.632065	0.565882
Minstral 3 8B Instruct	mistral.ministral-3-8b-instruct	0.707314	0.724719	0.137576	0.231252
Qwen3 32B	qwen.qwen3-32b-v1:0	0.709248	0.545294	0.549947	0.547611
Qwen3 Next 80B A3B	qwen.qwen3-next-80b-a3b	0.641451	0.462106	0.734803	0.56739

Comparison to prior baselines. On F1, Minstral 14B (0.566) and Qwen3 Next 80B A3B (0.567) slightly exceed head-only ModernBERT (test F1 0.555) and one-shot original-only prompting (test F1 0.497). Qwen3 32B is close (F1 0.548). Minstral 8B reaches the highest accuracy (0.707) but is very conservative on the remove class (recall 0.138; F1 0.231), so it is a poor moderation baseline despite high accuracy. Qwen3 Next 80B has the highest remove recall (0.735) at the cost of lower precision (0.462). Overall, the mid/large open-weight Bedrock models are competitive with ModernBERT on this blinded mirror-view task without fine-tuning; the 8B Minstral model is not.

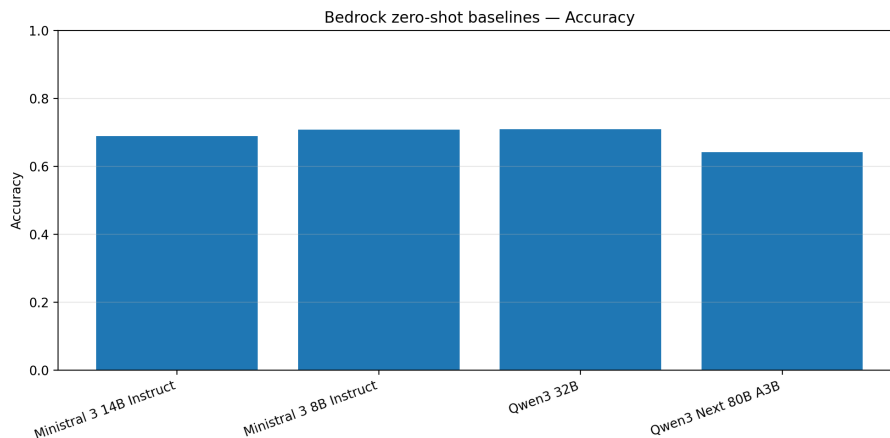


Figure 5: Bedrock zero-shot baseline accuracy

Step 2: LoRA fine-tuning I’m thinking we use two models:

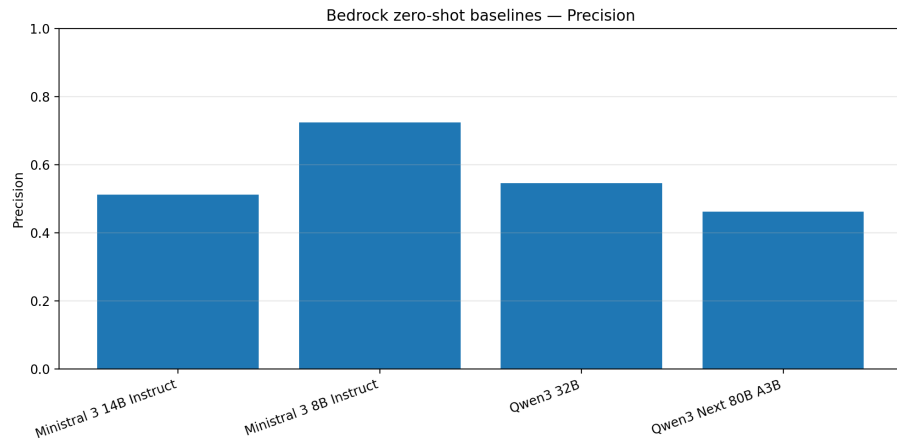


Figure 6: Bedrock zero-shot baseline precision

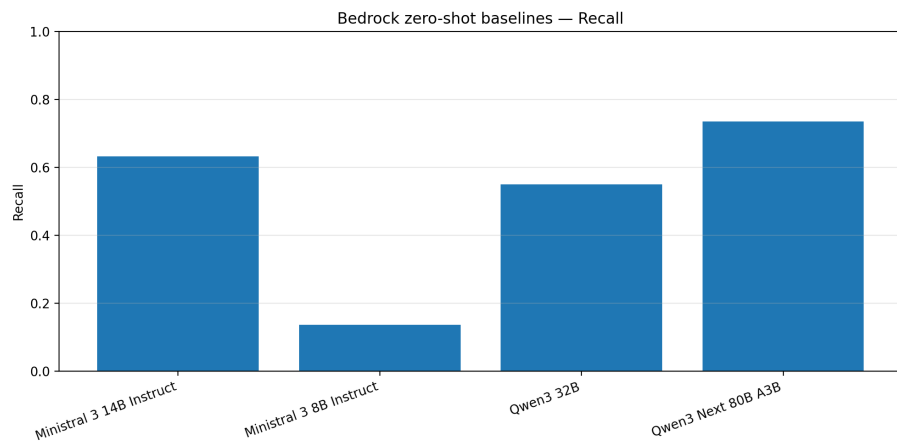


Figure 7: Bedrock zero-shot baseline recall

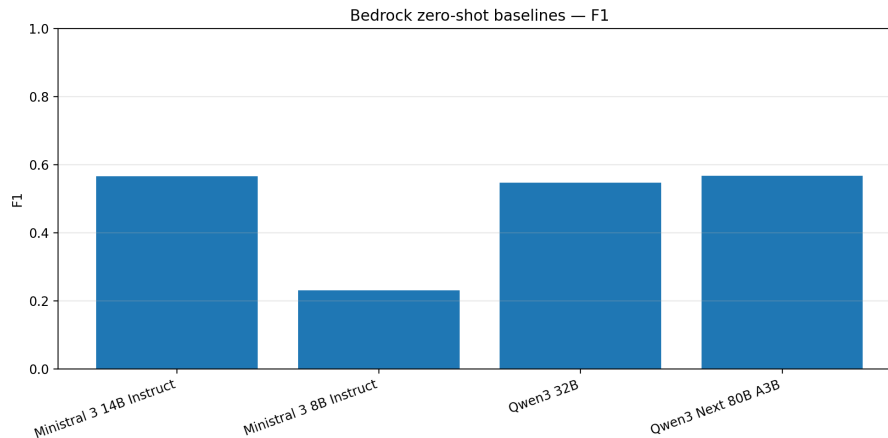


Figure 8: Bedrock zero-shot baseline F1

- `mistral.ministral-3-8b-instruct`
- `mistral.ministral-3-14b-instruct`
- `Qwen/Qwen3-8B`
- `Qwen 30B`

We can use QLoRA so that it fits comfortably on an H100.

Let's use Sagemaker Training Jobs + HuggingFace. We'll track the experiments using Weights and Biases.

We'll use QLoRA. We'll also shuffle the LoRA ranks between 16, 32, and 48.

The basic setup would be something like running a SageMaker training job with the proper configurations.

- Pulls the model from Hugging Face
- Reads train/test/validation datasets from S3
- Runs fine-tuning
- Streams metrics to Weights & Biases
- Saves LoRA adapter on the SageMaker instance and then uploads to S3.

We'll put this in `experiments/predict_keep_remove_2026_07_01/models/llm_finetuning/finetuning/`

I'm thinking a setup of something like this:

```
{base path}/finetuning/
models/
  {qwen 8b, mistral, etc}/
    config.py
    runner.py
    outputs/
```



```

{timestamp}/
  metrics.json
  outputs.csv
  metadata.json

```

```

runner.py
dataloader.py
metrics.py

```

We'll use the following:

TRL: HuggingFace's post-training library. We'll be using `SFTTrainer`. For adapter training, it integrates with PEFT.

The TRL part can look something like this:

```

from trl import SFTConfig, SFTTrainer

training_args = SFTConfig(
    output_dir=args.output_dir,
    num_train_epochs=3,
    max_seq_length=2048,
    report_to=["wandb"],
    ...
)

trainer = SFTTrainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["validation"],
    peft_config=peft_config,
    processing_class=tokenizer,
)

trainer.train()

```

TRL helps with the experimental scaffolding around fine-tuning, such as reading examples, applying chat/instruction formatting, tokenizing, loss, evaluation loop, and logging. TRL is also what's used to run PEFT on the models itself.

PEFT (“Parameter-efficient fine tuning”) is a library for adapting large pretrained models without having to fine-tune all model parameters. PEFT is the general library, while LoRA is one PEFT method.

For example, this might be how it looks:

```

from peft import LoraConfig

peft_config = LoraConfig(

```

```

r=16,
lora_alpha=32,
lora_dropout=0.05,
bias="none",
task_type="CAUSAL_LM",
target_modules=[
    "q_proj",
    "k_proj",
    "v_proj",
    "o_proj",
    "gate_proj",
    "up_proj",
    "down_proj",
],
)

```

For our task, let's include both attention and MLP modules as target modules for the LoRA.

We can first try LoRA, and then try QLoRA, to see what the difference in performance is.

I'm thinking that we can do ablations and testing on one model, and then run on the rest. Let's start with Qwen 3 8b.

Ablations:

- Qwen 3 8b: baseline (done in Step 1).
- Qwen 3 8b: LoRA fine-tuning (across $r=16$, 32, and 48).
- Qwen 3 8b: LoRA (bfloat16) vs. QLoRA (nf4).

Once we've got a set of optimal parameters, let's train the Mistral + Qwen 30B models.